

MOVIE WATCHLIST

Mobile Application Development Project

Student Name	Kirezi Hirwa Aldo
Registration Number	2401000842
Course	Mobile Application Development
Project	Project 10
Application	Movie Watchlist
Date	19 September 2026

University of Kigali – Musanze Campus

1. Introduction

Movie Watchlist is a Flutter mobile application designed to help users organize movies they want to watch. The application allows users to view movies, open movie details, add new movies, mark movies as watched, remove movies, and keep their movie information saved locally.

The application is intended for people who want a simple way to organize their personal movie collection. It demonstrates important mobile application development concepts including Flutter widgets, Dart programming, Riverpod state management, navigation, form validation, animations, JSON serialization, and local persistence.

The project also demonstrates how a mobile application can provide useful functionality through a clean interface while maintaining application state between sessions.

2. Problem Statement

Movie lovers may have difficulty remembering movies they want to watch and keeping track of which movies they have already watched. Using informal notes or relying only on memory can make the collection difficult to manage.

The Movie Watchlist application addresses this problem by providing a simple digital watchlist where users can store movie information, view details, mark movies as watched, and maintain the collection using local storage.

3. Objectives

General Objective

To develop a functional Flutter mobile application that allows users to manage and organize a personal movie watchlist.

Specific Objectives

- Create a user-friendly movie watchlist interface.
- Implement navigation between the movie list and movie detail screens.
- Allow users to add movies through a validated form.
- Allow users to mark movies as watched and remove movies.
- Use Riverpod for application state management.
- Use JSON serialization and SharedPreferences for local persistence.
- Implement Flutter animations including Hero and implicit animations.

4. Tools and Technologies

Technology	Purpose
Flutter	Used to build the mobile application interface and application structure.
Dart	Programming language used to implement the application logic.
Riverpod	Used for state management and sharing movie data between screens.
SharedPreferences	Used to store movie information locally on the device.
JSON	Used for serialization and deserialization of movie data.
Git	Used for version control and milestone commits.
GitHub	Used to host and submit the project repository.
Android Studio / Android SDK	Used for Android development and testing.
BlueStacks / Android Emulator	Used to test the application on an Android environment.

5. Application Design

Home Screen: Displays movies in a grid and provides access to the add-movie functionality.

Movie Detail Screen: Displays the selected movie's poster, title, genre, year, description, and watched status.

Add Movie Screen: Provides a validated form for entering a movie's title, genre, year, poster URL, and description.

Navigation: Users move from the Home Screen to the Movie Detail Screen by selecting a movie. The Add Movie button opens the movie form.

Data Model: Each movie contains an ID, title, poster URL, description, genre, year, and watched status.

State Management: MovieNotifier manages the movie list using Riverpod. Changes such as adding, removing, or marking movies as watched update the application state and are saved locally.

M1: Project Setup and Custom Launcher Icon

During M1, the Flutter project was created and the basic project structure was prepared. Required dependencies were configured and a custom launcher icon was added. This milestone helped establish the foundation of the application and introduced the process of configuring a Flutter project before implementing application features.

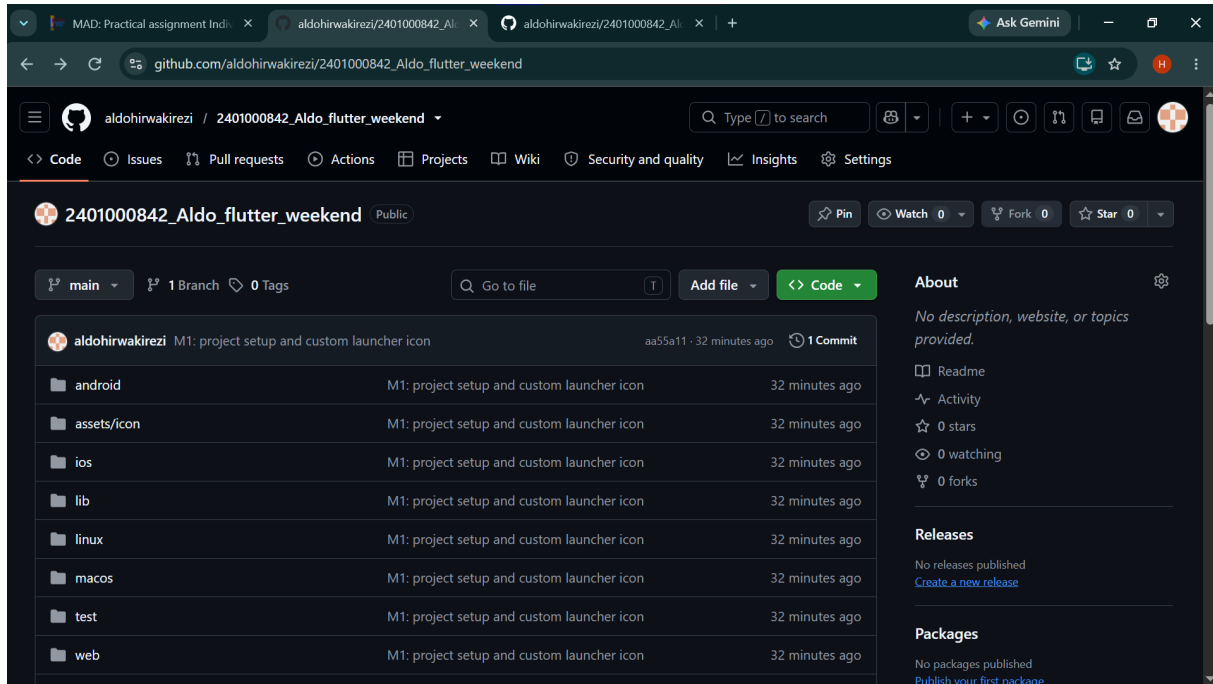


Figure M1: Flutter project setup, environment verification, and custom application icon.

M2: Movie Model

During M2, the Movie model was created. The model contains the information required by each movie, including its ID, title, poster URL, description, genre, year, and watched status. JSON conversion methods were also implemented. This milestone helped demonstrate how Dart classes represent structured application data.

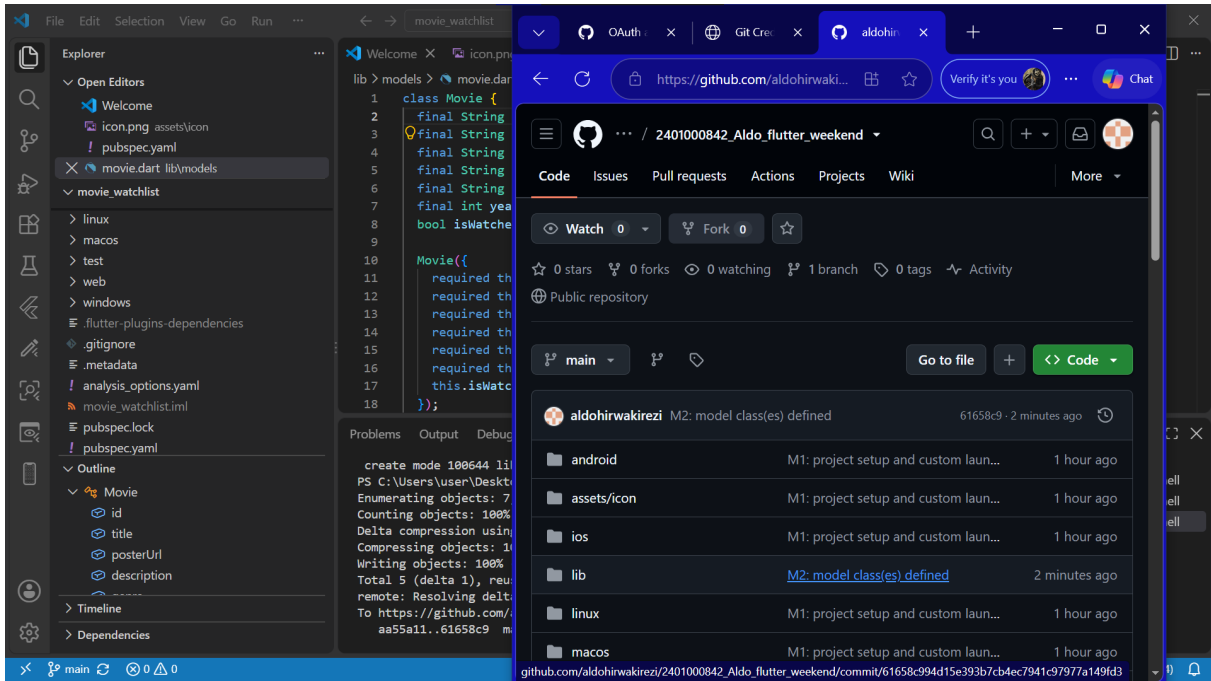


Figure M2: Movie model and JSON serialization implementation.

M3: Riverpod Provider

During M3, Riverpod was introduced for state management. MovieNotifier was implemented to manage the movie list and provide operations for adding, removing, and updating movies. This milestone helped demonstrate how application state can be managed independently from the user interface.

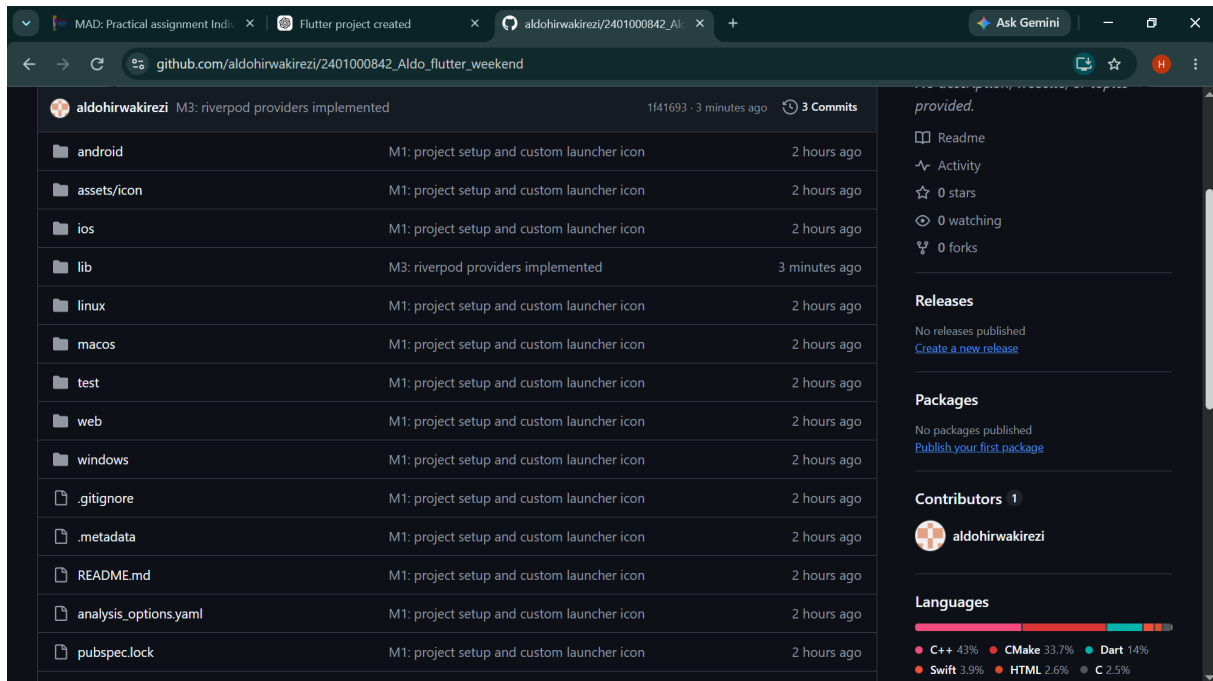


Figure M3: Riverpod MovieNotifier and movie provider.

M4: Home Screen and Empty State

During M4, the first major user interface was created. Movies were displayed using a GridView and an empty state was provided when no movies were available. This milestone helped develop skills in Flutter layouts, widgets, and conditional user interface rendering.

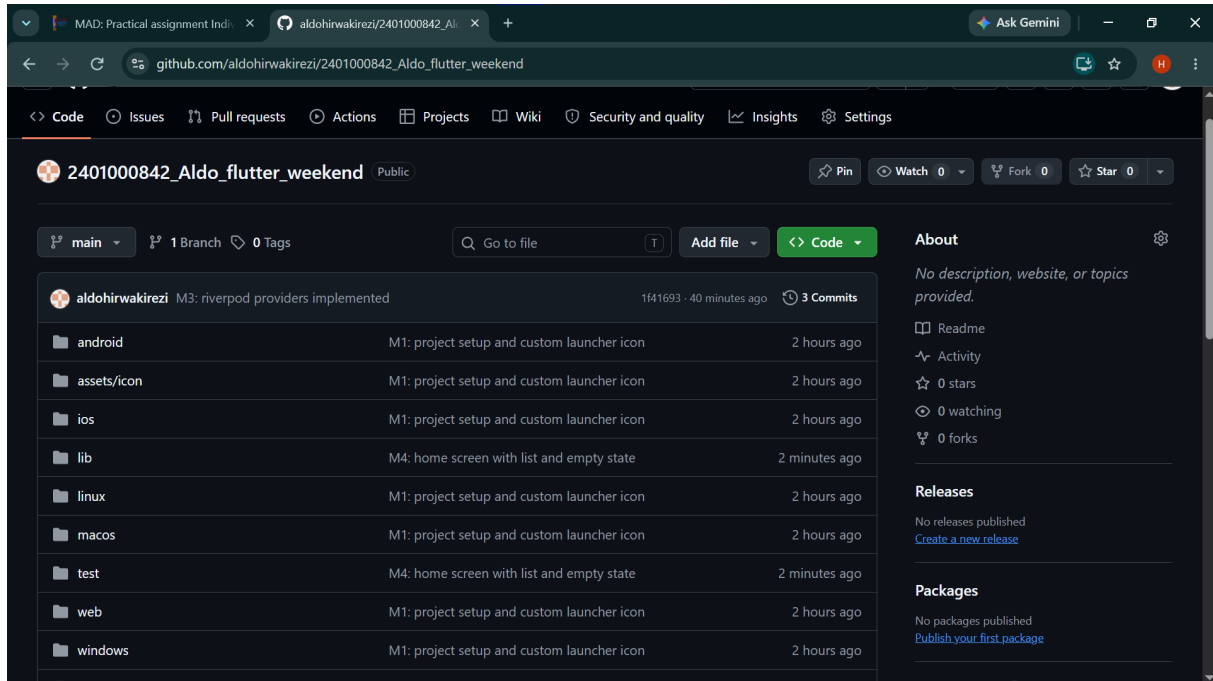


Figure M4: Movie grid and empty-state interface.

M5: Movie Detail Screen and Navigation

During M5, a second screen was created to display complete movie information. Navigation was implemented so that a selected Movie object could be passed from the home screen to the detail screen. This milestone helped demonstrate Flutter navigation and passing data between screens.

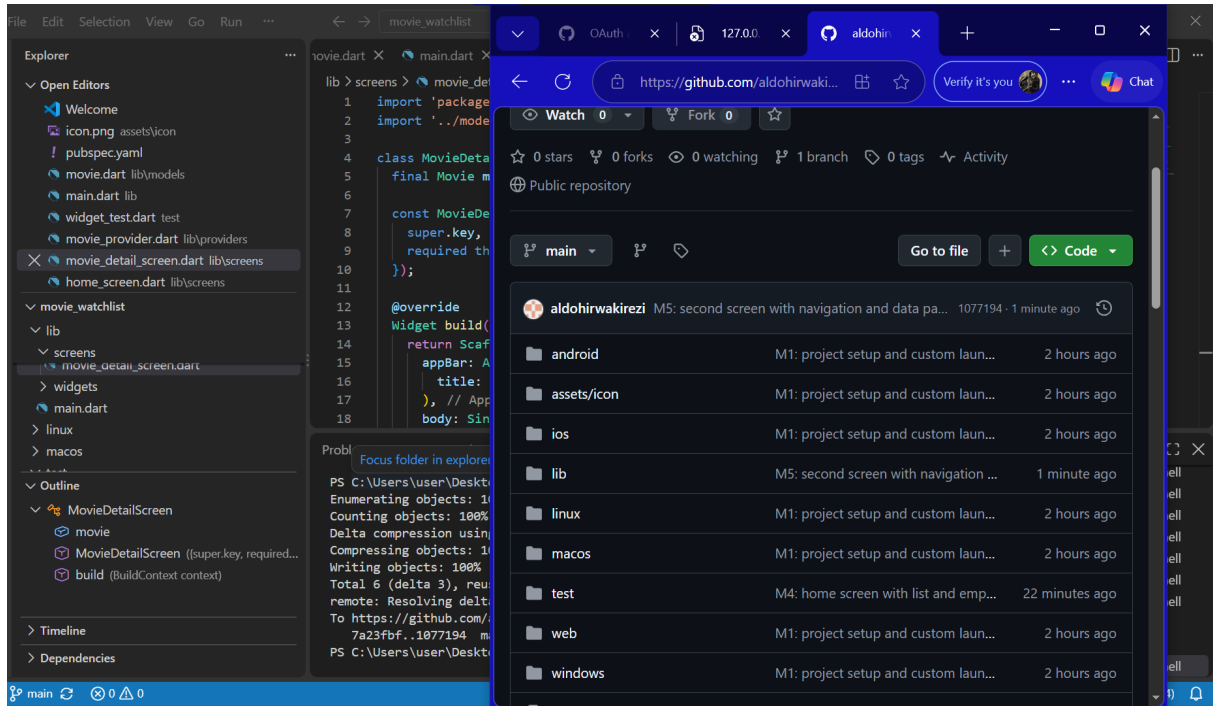


Figure M5: Movie detail screen and navigation from the movie collection.

M6: Animations

During M6, the application interface was improved with AnimatedContainer, AnimatedOpacity, and Hero animations. These animations provide visual feedback and smoother transitions when users interact with movie cards and open details. This milestone improved understanding of Flutter's animation capabilities.

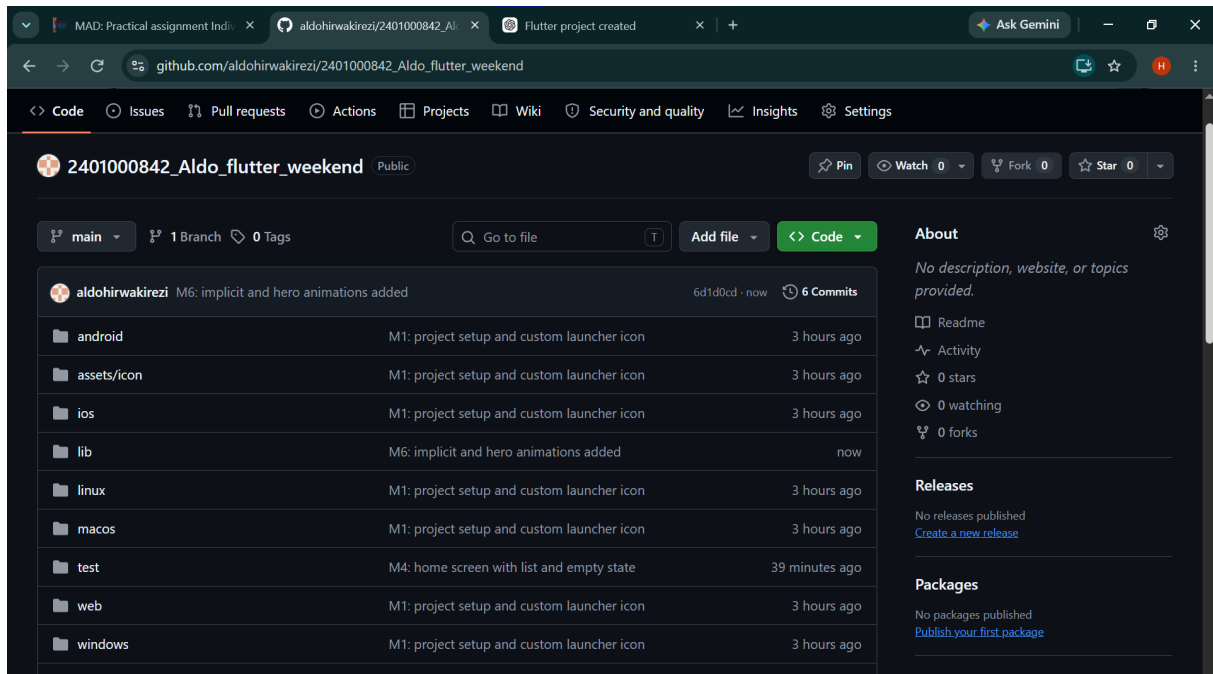


Figure M6: Animated movie cards and Hero transition.

M7: Persistence, Validation and Error Feedback

During M7, local persistence was implemented using SharedPreferences. Movie data is converted to JSON before being stored and reconstructed when the application loads. The add-movie form also validates user input and displays feedback using a Snackbar. This milestone demonstrated the importance of data persistence and input validation.

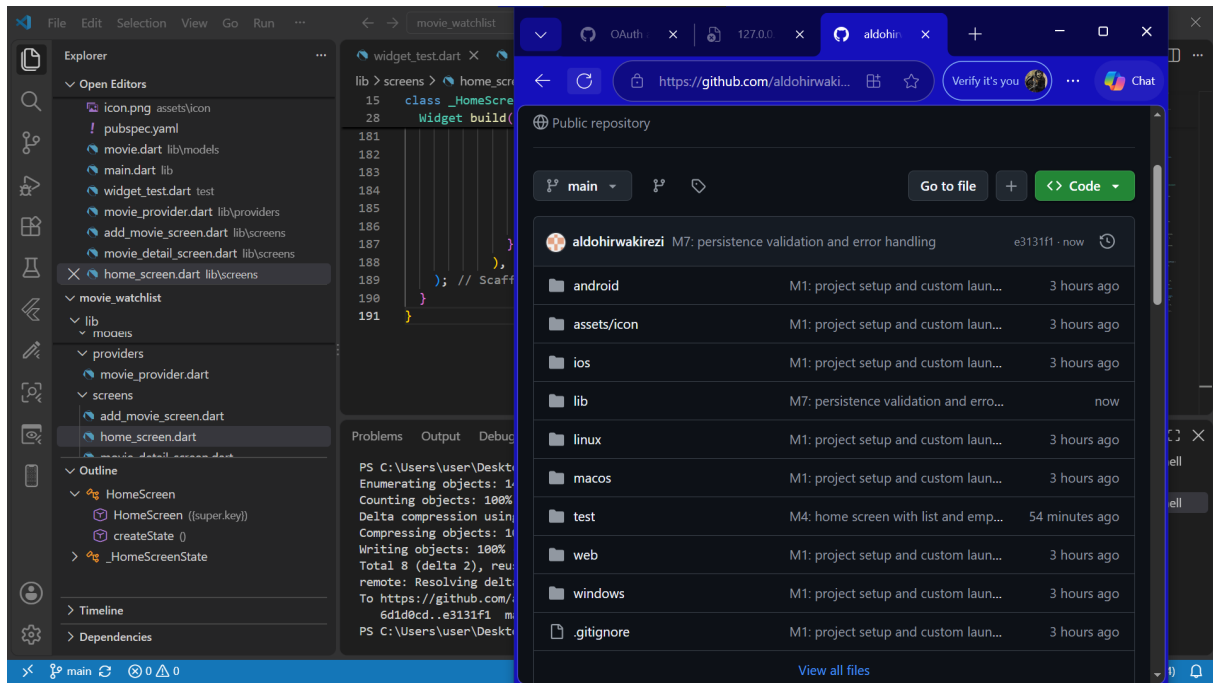


Figure M7: Add movie form, validation, local persistence, and user feedback.

M8: Final Polish and Release Build

During M8, the application was tested and prepared for final submission. The project was analyzed, the release APK was generated, and the final application was tested in an Android environment. This milestone helped demonstrate the complete process from development to application delivery.

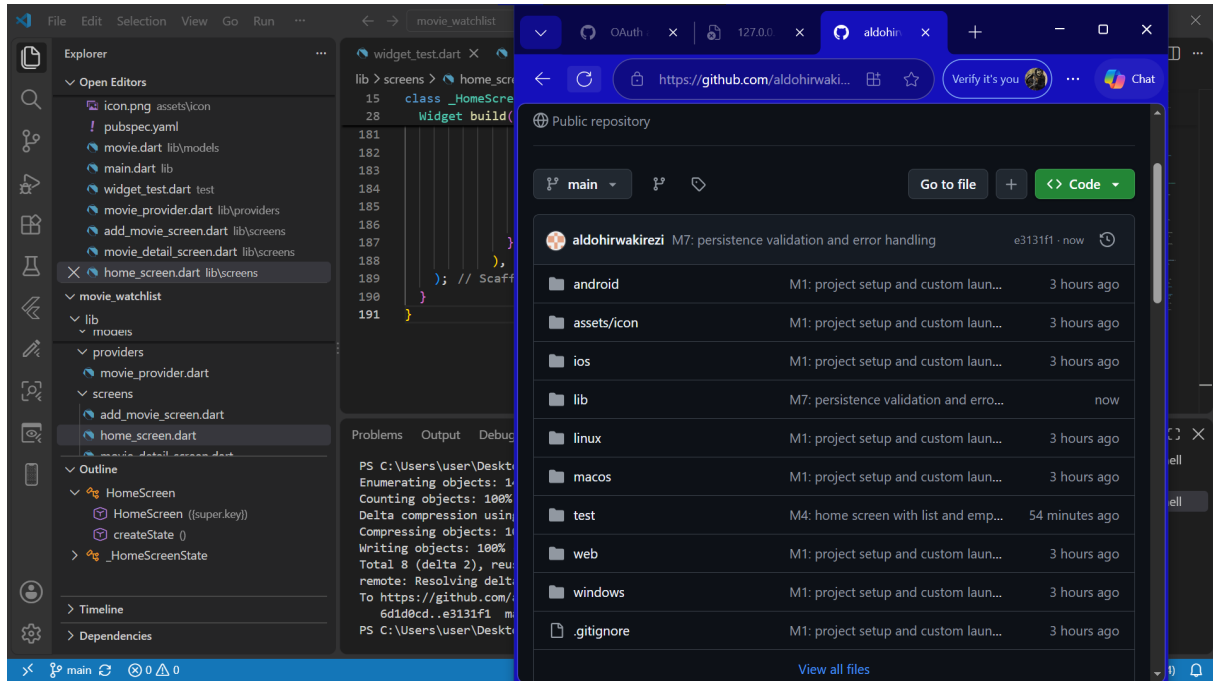


Figure M8: Final Movie Watchlist application and release build.

7. Testing

Test	Result
Application launch	Application launched successfully.
Movie list	Movies were displayed correctly on the home screen.
Empty state	Empty-state interface was displayed when there were no movies.
Movie details	Selecting a movie opened its detail screen.
Add movie	A new movie could be added through the form.
Validation	Invalid form data was rejected with validation messages.
Watched status	Movies could be marked as watched.
Remove movie	Movies could be removed from the collection.
Persistence	Saved movie information remained available after restarting.
Animations	Hero and implicit animations were visible during interaction.
Release APK	The release APK was successfully generated and tested.

8. Challenges Encountered

State management: Managing movie data across different screens was initially challenging. Riverpod was introduced to provide a central state management solution.

Data persistence: Keeping movie information after restarting the application required local storage. SharedPreferences and JSON serialization were used to solve this.

Form validation: User input needed to be checked before adding a movie. Validation rules were added for the required fields and poster URL.

Animations: Implementing Hero and implicit animations required understanding how widgets change state and how animation tags must match between screens.

9. Limitations

- The application does not provide user accounts or cloud synchronization.
- Movie information is entered manually rather than being retrieved from a full movie database API.
- The application depends on valid poster image URLs for online movie posters.

10. Conclusion

The Movie Watchlist project successfully demonstrates the development of a functional Flutter mobile application. The application combines a structured movie model, Riverpod state management, navigation, form validation, animations, JSON serialization, and local persistence.

The project also provided practical experience in using Git and GitHub, testing a Flutter application on an Android environment, and producing a release APK for submission.

In a future version, the application could be extended with a movie database API, user accounts, cloud synchronization, search, categories, and additional movie information.

11. References

- Flutter Documentation – Flutter framework and widgets.
- Dart Documentation – Dart programming language.
- Riverpod Documentation – State management for Flutter and Dart.
- SharedPreferences Documentation – Local key-value storage.
- Git Documentation – Version control.
- GitHub Documentation – Repository hosting and collaboration.

Project Repository

https://github.com/alдохirwakirezi/2401000842_Aldo_flutter_weekend